WEEK 7

Distributed Logs & Apache Kafka

From batch files to real-time event streams

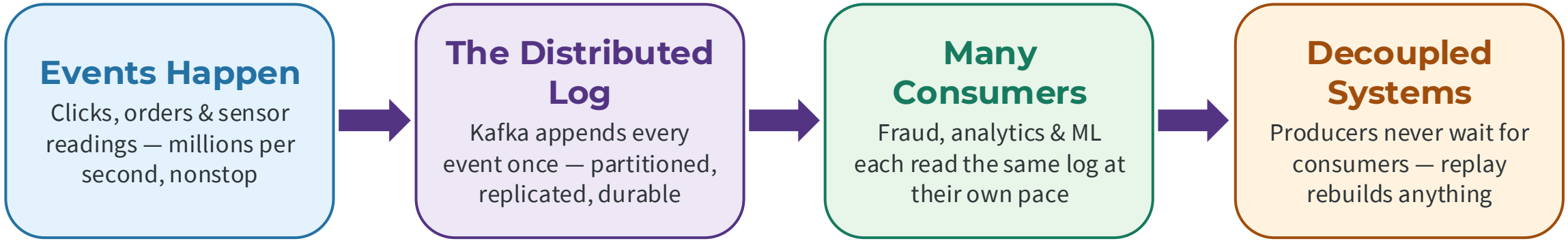
The Log Abstraction
Append-only, ordered, replayable

Kafka Core
Topics, partitions & consumer groups

Semantics & Schemas
Delivery, compaction & schema registry

CDLE • MEIC/MEIM • ISEL

From Events to Every Consumer



This is the log abstraction

One durable, ordered record of everything that happened — the backbone that turns data into real-time event streams.

NEW TERMS

- **`Kafka`** open-source distributed log platform
- **`Distributed log`** append-only log across servers
- **`Producer`** a program that writes events
- **`Consumer`** a program that reads events
- **`Offset`** a record's position in the log
- **`Replay`** re-read events from an earlier offset

Contents

Everything for week 7 — the log abstraction, Kafka's core machinery, delivery semantics & schemas

01 The Log Abstraction

Why point-to-point pipelines break
From write-ahead logs to Kafka
The log: append-only, ordered, replayable
Queue vs log

02 Kafka Core

Topics, partitions & offsets
Producers, keys & ordering
Consumer groups & rebalancing
Replication, leaders & the ISR

03 Semantics & Schemas

Producer acks & idempotence
At-most / at-least / exactly-once
Retention & log compaction
Schema management & the registry
Reading list: primary & secondary

04 Lab — Week 7

Producing & consuming event streams
AI-assisted event-schema design review
Your week-7 checklist
Next week: stream processing engines

Scenario — Clicks, in Real Time

The same clickstream as week 1 — but now four teams need it, and one needs it in seconds

The Question

"Every team wants the clickstream — and the fraud team wants it within seconds."

- The clickstream: ≈50 000 events/s — fraud, recommendations, analytics & audit all need it
- The nightly batch answers **`tomorrow`** — fraud detection needs seconds
- Point-to-point pipes multiply — every new consumer rewires every producer
- And every copy must agree on what happened, and in what order

The Integration Math

```
# point-to-point:
#   5 producers × 4 consumers = 20 pipes
#   each with its own format, retries & lag

# one shared log: 5 + 4 = 9 connections
the catch — the log must:
scale writes across partitions,
  keep order where it matters,
  survive broker failures,
  and let every reader replay = hard
→ that is what Kafka does
```

The lesson

Stop wiring systems to each other — wire them all to one shared, ordered, replayable log. That is Kafka's founding idea, and this week's subject.

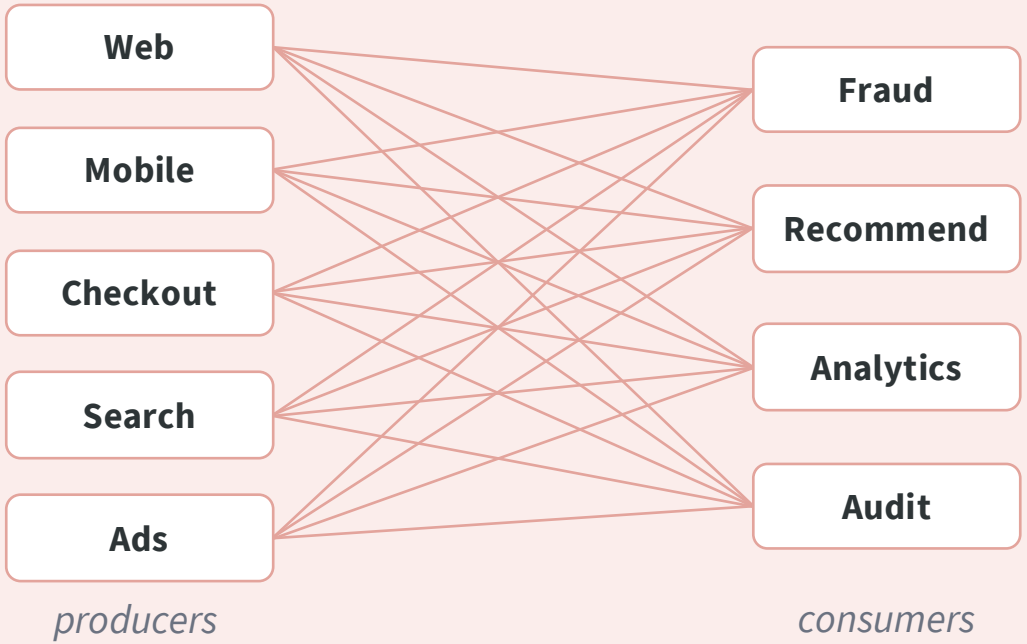
NEW TERMS

- **`Point-to-point`** one pipe per producer–consumer
- **`Nightly batch`** a once-a-day processing job
- **`Lag`** how far a consumer trails the newest event

Point-to-Point vs One Shared Log

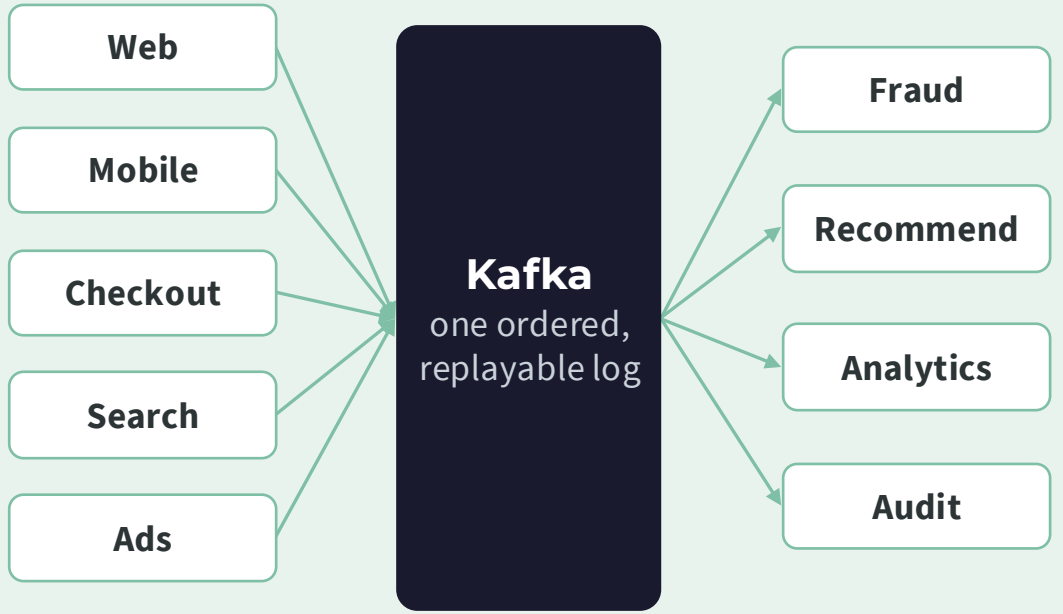
Five producers and four consumers: 20 custom pipes, or 9 connections to one log

Point-to-point — every pair wired



20 pipes — each with its own format, retries & lag

Shared log — everyone wires to Kafka



9 connections — one format, one order, replay included

A fifth consumer adds 5 more pipes point-to-point, but only 1 connection to the log.

The Log Grows Up

The oldest idea in databases, promoted to the system of record between systems

1

1970s
The write-ahead log

Databases append every change to a log before applying it — recovery and replication fall out of an ordered, durable record.

2

2011
Kafka at LinkedIn

Kreps, Narkhede & Rao publish Kafka: the log becomes shared infrastructure — partitioned, replicated, and open source.

3

2015–16
An ecosystem forms

Kafka Connect, Kafka Streams and the Schema Registry turn the log into a full event-streaming platform.

4

2025
The event backbone

Kafka 4.0 manages its own metadata (KRaft, no ZooKeeper); exactly-once and tiered storage are mature. It carries the events that feed lakehouses and ML.

Why this matters

- Kafka did not invent the log — it distributed it: partitioned for scale, replicated for safety, retained for replay.
- Optional reading: the original Kafka paper (Kreps et al., 2011) — short, and the source of every idea this week.

NEW TERMS

- **`Write-ahead log (WAL)`** log changes, then apply them
- **`KRaft`** Kafka's consensus; replaced ZooKeeper
- **`Exactly-once`** each event takes effect once
- **`Tiered storage`** old data moved to cheap storage
- **`Connect · Streams`** move data · process streams
- **`Schema Registry`** stores and checks schemas

The Log — Three Properties

1 — Append-only
writes only at the end, immutable

2 — Ordered
every record gets an offset

3 — Replayable
re-read from any offset, any time

Three properties, one structure

```
// the log — a structural view
records are appended at the tail — never edited in place  // 1
each gets an offset — its permanent position  // 2
  reads are non-destructive — history is kept  // 3
  → consumers rewind & replay

// what the properties buy you
order — one truth about event sequence, per partition
fan-out many readers, one write — zero coupling
  crash? resume from the last committed offset — no loss
  → the log is the source of truth
```

Why these three together

- ✓ Append-only writes are sequential — and fast
- ✓ Offsets give every reader an exact position
- ✓ Replay turns bugs into re-runs, not outages
- ✓ New consumers bootstrap from full history
- ✓ Databases, WALs & Kafka share this same core

A log is the simplest structure that gives ordering, durability and fan-out at once — everything else in Kafka is machinery around it.

NEW TERMS

`Immutable` a written record is never changed
`Non-destructive read` the record stays in place
`Fan-out` one write, read by many consumers
`Committed offset` the last position saved

Queue vs Log

Both move messages — but they answer 'who reads, and what remains?' very differently

Queue — destructive read

```
# one message → ONE worker
msg = queue.pop()
process(msg)  # gone
# no history, no replay
# readers split the traffic
```

Log — replayable read

```
# every reader sees all
rec = log.read(offset)
process(rec)  # kept
offset += 1  # reader moves on
offset = 0   # rewind for replay
```

The practical difference

```
            queue   vs log
read      = pop     vs seek
fan-out   = one     vs all groups
history   = none    vs retained
scaling   = workers vs partitions
```

✓ **Many independent consumers**

✓ **Replay after bugs & crashes**

✓ **Order preserved per partition**

*Kafka is a log, not a queue. **Queues** excel at task dispatch; **logs** excel at event streams shared by many systems. Kafka can still split work across workers like a queue, without losing history.*

NEW TERMS

- **`Queue`** one worker per message, then gone
- **`read(offset)`** fetch the record at that position
- **`offset = 0`** rewind and replay from the start
- **`Task dispatch`** handing jobs to workers

Key Concepts

```
# checkout service · producer
p = KafkaProducer(bootstrap_servers="kafka:9092")
p.send("clicks", key=b"user-42",
       value=b'{"page": "/checkout"}')
p.flush()   # send() only buffers
# hash(key) % partitions → same partition → in order
```

```
# fraud service · consumer
c = KafkaConsumer("clicks",
        bootstrap_servers="kafka:9092",
        group_id="fraud",
        auto_offset_reset="earliest")
for msg in c:
    print(msg.partition, msg.offset,
          msg.key, msg.value, msg.timestamp)
# a group shares partitions; offsets track progress
```

The Vocabulary

- **Topic** (`"clicks"`) → a named, partitioned stream of events
- **Partition** (`msg.partition`) → one ordered log of a topic
- **Offset** (`msg.offset`) → a record's position in its partition
- **Broker** (`"kafka:9092"`) → one server of the cluster
- **Consumer group** (`"fraud"`) → consumers sharing partitions
- **Retention** (`7 days`) → how long the log keeps its events

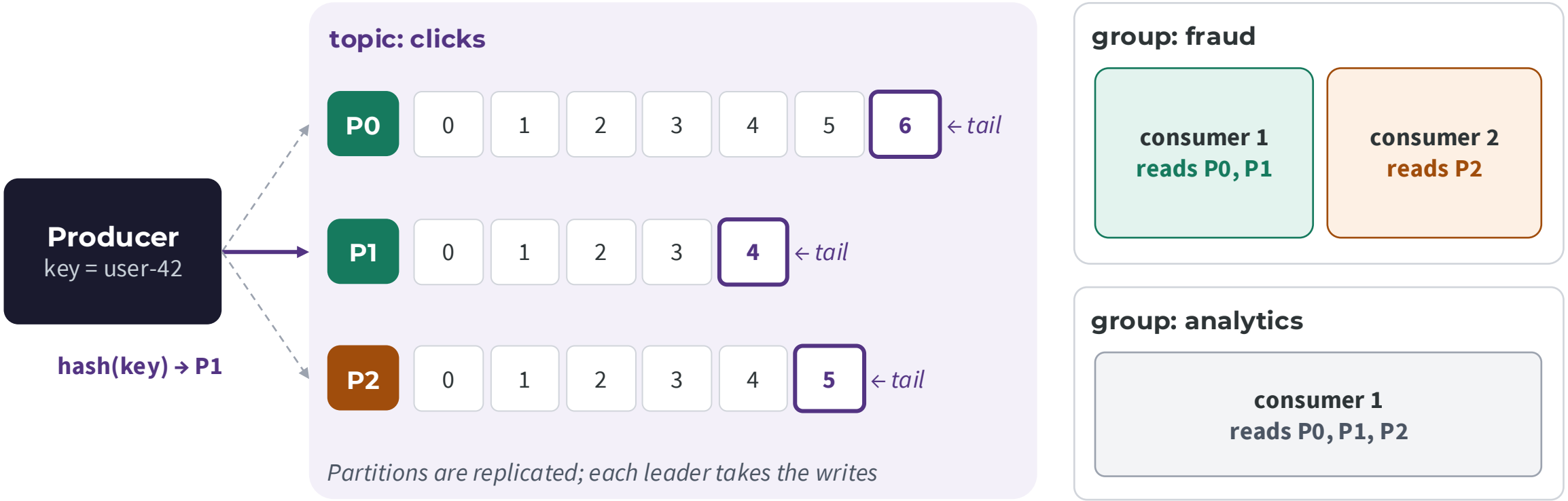
You will write exactly this code in today's lab — first, the machinery these six words describe.

NEW TERMS

`bootstrap_servers` brokers a client calls first
`key` · `value` routing key and payload
`group_id` the consumer group a consumer joins
`send()` · `flush()` queue a record · send queue to brokers, wait

Anatomy of a Topic

One topic, three partitions, two consumer groups: each group reads every event



Within a group, each partition has exactly one reader; across groups, every group gets every event.

NEW TERMS

- `hash(key)` maps each key to one fixed partition
- `Tail` the end of the log, where records are added
- `Replica` a copy of a partition on another broker (multi-broker)
- `Leader` the replica that takes all writes

The Rules of the Log

ORDERING

Per partition only

Order is guaranteed within a partition — never across a whole topic

KEYS

Same key, same partition

hash(key) routes records — one entity's events stay in sequence

PARALLELISM

Partitions scale reads & writes

The unit of scaling — more partitions allow more concurrent writers & readers

GROUPS

One reader per partition

Within a group, each partition is assigned to exactly one consumer

REBALANCING

Membership changes

A consumer joining or failing triggers partition reassignment

REPLICATION

Leader & followers

Every partition is copied across brokers; writes go to the leader

THE ISR

In-sync replicas (ISR)

With acks=all, a write is confirmed once every in-sync replica has it

OFFSETS

Committed progress

Committed offsets let a group resume exactly where it stopped

Eight rules, one mental model: partitioned, keyed, replicated logs read by coordinated groups.
Every Kafka design question reduces to these.

NEW TERMS

`Rebalancing` reassigning partitions in a group

`Follower` a replica that copies the leader

`ISR` acks=all, followers fully caught up with the leader; acks=1, wait for the leader only; acks=0, no wait at all

Replication & the ISR

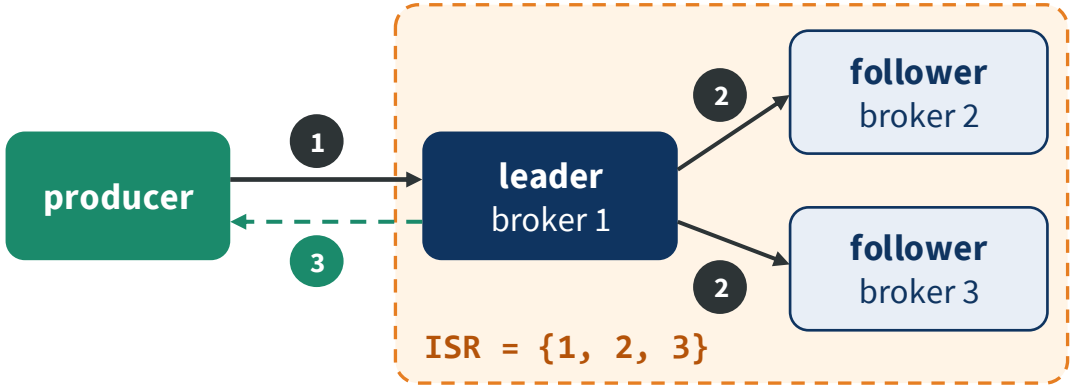
With replication factor 3, every partition has one leader and two followers on other brokers; a write counts once the in-sync replicas have it

Where the replicas live

broker 1	broker 2	broker 3
P0 leader	**P0** follower	**P0** follower
P1 follower	**P1** leader	**P1** follower
P2 follower	**P2** follower	**P2** leader

Leaders are spread across brokers, so every broker takes a share of the writes.

One write to P0 with acks=all



1. producer writes to the leader
2. followers copy it from the leader
3. leader acks once the whole ISR has it

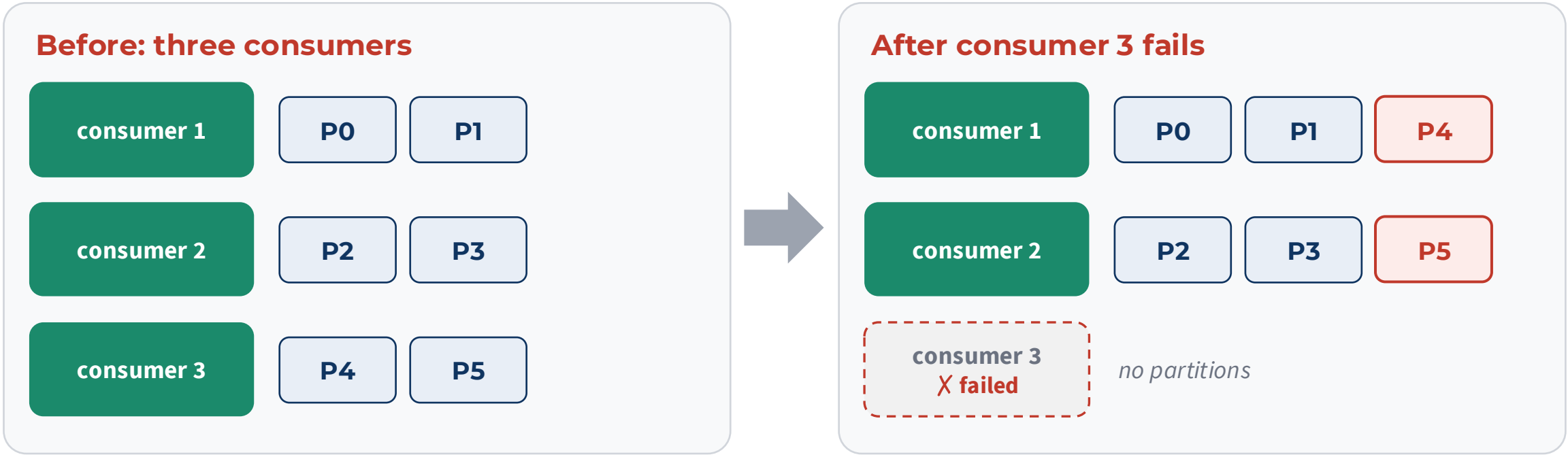
If broker 1 dies, the controller promotes a follower from the ISR to leader: no acknowledged write is lost.

NEW TERMS

- **`Controller`** broker that assigns partition leaders
- **`ISR`** In-sync replicas
- **`Failover`** a follower takes over as leader

Rebalancing a Consumer Group

When a consumer stops sending heartbeats, the coordinator hands its partitions to the members that are left



Group: fraud · topic: clicks · 6 partitions. Moved partitions in red resume from their last committed offset.

The group pauses while partitions move, so frequent rebalances show up as consumer lag.

NEW TERMS

- **`Heartbeat`** a consumer's periodic 'still alive' signal
- **`Group coordinator`** broker that tracks a group's members

The Producer's Durability Dial

Every write travels producer → leader broker → follower brokers; acks sets how far the producer waits

✓ What the producer controls

- **acks** — how many replicas must confirm each write
- **Idempotence** — retries can no longer create duplicates (requires acks=all)
- **Batching** — linger & batch size trade latency for throughput
- **Retries** — transient broker errors handled automatically
- **Transactions** — atomic writes across topics & partitions

acks=0 fire & forget	acks=1 leader ack only	acks=all in-sync replicas ack
DURABILITY ✗ None	DURABILITY ✗ Leader only	DURABILITY ✓ Strongest
LATENCY ✓ Lowest	LATENCY ✓ Low	LATENCY ✗ Highest
THROUGHPUT ✓ Highest	FAILOVER ✗ Can lose data	DEFAULT ✓ Since 3.0

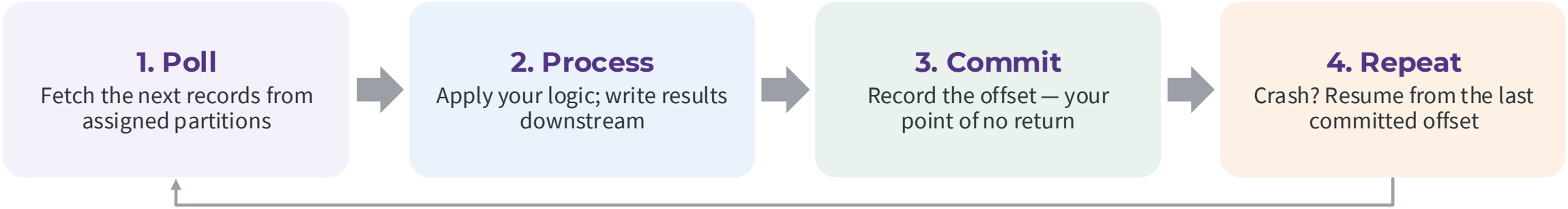
One producer setting decides whether a dead broker means lost events. Set acks per producer, to match what its data is worth.

NEW TERMS

- `acks` replicas that must confirm each write
- `Idempotence` retries never duplicate a write
- `Transaction` writes that succeed or fail together
- `linger.ms` · `batch.size` batch wait and size
- `Throughput` events handled per second

One Loop for Every Consumer

Poll, process, commit — and where you commit decides your delivery guarantee



the loop — three delivery semantics

```
commit first  → at-most-once  · crash = events lost
process first → at-least-once · crash = duplicates
exactly-once: idempotent sink or transactions
  read_committed:
      "consumers skip aborted transactional writes"
rule: make consumers idempotent anyway
// the guarantee is end-to-end, not a flag
```

Failure modes to avoid

- **Commit-then-crash** — events silently dropped
- **Process-then-crash** — duplicates without idempotence
- **Poison pills** — one bad record wedges the loop
- **Rebalance storms** — slow loops, endless reassignment

NEW TERMS

`Commit` save the consumer's offset to Kafka
`At-most-once` no duplicates; may lose events
`At-least-once` no loss; may duplicate
`read_committed` skip aborted transactions
`Idempotent sink` repeat writes change nothing

Commit Order & Crashes

The same crash, two commit orders: commit first and events are lost, process first and events repeat

Commit first
at-most-once

STEP 1	STEP 2	STEP 3	RESTART
poll	**commit**	**process 10–12**	**resume at 15**
offsets 10–14	next offset = 15	✗ crash	13, 14 lost

Process first
at-least-once

STEP 1	STEP 2	STEP 3	RESTART
poll	**process 10–12**	**commit**	**resume at 10**
offsets 10–14	✗ crash	never runs	10–12 run twice

Atomic commit
exactly-once

Exactly-once: the output and the offset commit land together, in one transaction or through an idempotent sink, so a restart neither skips nor repeats.

Most pipelines choose at-least-once and make consumers idempotent, so a repeat changes nothing.

Retention & Log Compaction

RETENTION
Delete by age or size
the log as a bounded history of recent **events**

- **Time-based** *retention.ms* drops closed segments older than the window — 7 days by default
- **Size-based** *retention.bytes* caps each partition — oldest segments deleted first (unlimited (-1) by default)
- **The contract** consumers slower than the window lose data — monitor lag against retention

COMPACTION
Keep latest per key
the log as an ever-current table of **state**

- **How it works** a background cleaner rewrites segments, keeping the newest value per key
- **Tombstones** a null value marks a key for deletion — how compacted topics forget
- **Use it for** changelogs, consumer offsets & state restore — replay yields current state

Golden rule: *retention bounds history; compaction bounds state.*
Compaction guarantees at least the latest value per key survives — it is a table in disguise, not a shorter history. Both policies can combine.

NEW TERMS

`Segment` one log file; only the newest takes writes
`retention.ms · .bytes` age and size limits
`Log compaction` keep the latest value per key
`Tombstone` a null value that deletes a key
`Changelog` a topic recording every state change
`Active segment` file being written; closed at 1 GB or 7 days

Log Compaction, Step by Step

The cleaner keeps the newest value for each key and drops the older ones

Before every write is kept, in order

offset 0	offset 1	offset 2	offset 3	offset 4	offset 5	offset 6	offset 7	offset 8
~~K1 = a~~	~~K2 = b~~	~~K1 = c~~	~~K3 = d~~	~~K2 = e~~	**K1 = f**	**K3 = null**	**K4 = g**	**K2 = h**

background cleaner rewrites old segments

After one record per key; offsets keep their numbers

	offset 5	offset 6	offset 7	offset 8
Offsets 0–4 are gone. Consumers skip the gaps; nothing is renumbered.	**K1 = f**	**K3 = null**	**K4 = g**	**K2 = h**

K3 = null is a tombstone: it deletes K3, and is itself removed after delete.retention.ms (1 day by default).

Replaying a compacted topic rebuilds the current state: K1 = f, K2 = h, K4 = g.
With invoices: key may be the invoice ID.

NEW TERMS

- `Cleaner` thread that compacts segments
- `delete.retention.ms` how long tombstones stay

The Schema Contract

Events outlive the code that wrote them — the schema is the API between producers and consumers

Why schemas matter

```
events — read years later, by teams you never met
formats — Avro, Protobuf, JSON Schema carry types
the registry stores & versions every schema
  /* producers register, consumers resolve */
evolution is checked against a compatibility mode
  /* breaking changes rejected at the door */
→ contracts enforced before damage is done
```

Compatibility modes

- BACKWARD — new schema reads old data (default)
- FORWARD — old consumers read new data
- FULL — both directions; safest for shared topics
- NONE — anything goes; every consumer at risk

One workflow for every schema change:

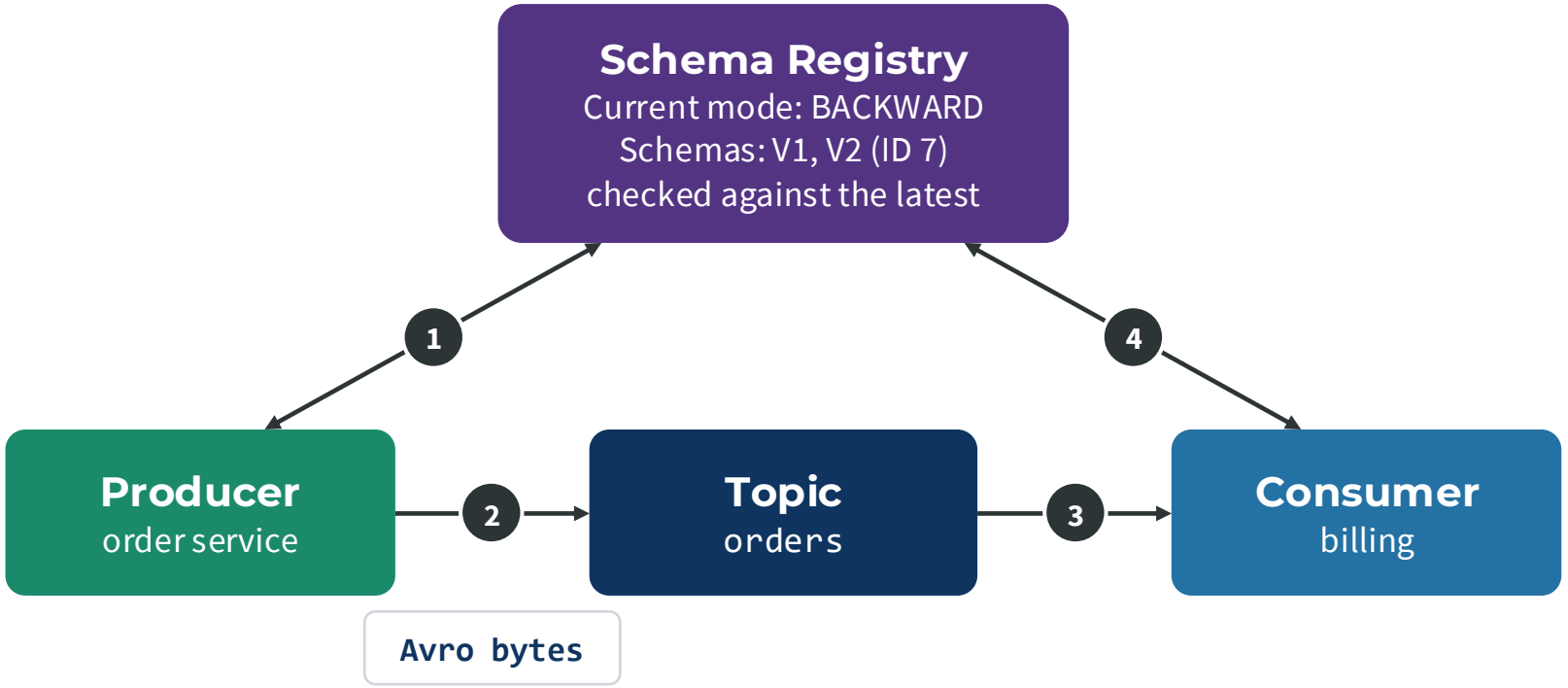
1 Propose — Draft the new version ▶ **2 Check** — Test it against the mode ▶ **3 Register** — Registry assigns an ID ▶ **4 Evolve** — Clients upgrade safely

NEW TERMS

- **Schema** an event's field names and types
- **Avro · Protobuf** typed binary formats
- **Schema evolution** changing a schema over time
- **Breaking change** an edit old readers can't read

How the Registry Guards a Topic

Producers register each schema version before sending; events carry only a schema ID, and consumers look it up



Looked up once
producers and consumers cache schemas by ID, so the registry is not on the hot path

What if: register V3 at ①
it adds a required field (no default); the producer gets an error and nothing reaches the topic
✗ **V3 rejected**

1. register V2 and get back schema ID 7
2. send events with ID 7 (instead of the schema)
3. the consumer reads the events
4. look up ID 7 to decode the bytes

NEW TERMS

`Schema ID` the number the registry assigns a version

`Schema cache` a client's local copy of looked-up schemas

`Hot path` code that runs for every event

What Each Mode Lets Through

A field with a default is optional; almost every rejected change involves a field without one

Mode	Guarantee	✓ Allowed	✗ Rejected	Upgrade first
BACKWARD (default)	**new** consumers **read old data**	delete a field; add an optional field	add a required field	consumers
FORWARD	**old** consumers **read new data**	add a field; delete an optional field	delete a required field	producers
FULL	both directions	add or delete optional fields	any change to a required field	either, any order
NONE	no check at all	anything, even type changes	nothing	all at once

Example: slide 20's V3 adds a required field, so BACKWARD rejects it. Give that field a default and the same V3 passes.

Example: in FORWARD, deleting an optional field is safe: old consumers fill it in with the default from their own (old) schema.

Each mode also has a `_TRANSITIVE` variant that checks every past version, not just the latest; use it when consumers replay from offset 0.

NEW TERMS

- **`Optional field`** a field with a default value
- **`Required field`** a field with no default
- **`Transitive`** checked against all past versions
- **`Upgrade order`** which side must deploy first

The Kafka Ecosystem at a Glance

The moving parts

Brokers + KRaft
the cluster and its consensus

Kafka Connect
source and sink connectors

Kafka Streams
processing inside your app

Schema Registry
contracts and compatibility

MirrorMaker
cross-cluster replication

Where each one fits



Store
Brokers persist partitioned, replicated logs on disk



Move
Connect streams data in from databases and out to lakes & indexes



Process
Streams, Flink & Spark consume topics and write results back



Govern
The registry enforces schemas; ACLs & quotas police the cluster

`You will run the core of this in the lab`: *broker, registry and clients live in this week's compose file. Connect & Streams return in weeks 8–9.*

NEW TERMS

- **`MirrorMaker`** copies topics between clusters
- **`Flink`** a stream-processing engine
- **`ACL`** rule: who may do what on a topic or group
- **`Quota`** a cap on a client's throughput
- **`Compose file`** config that starts the lab

Reading List

Ranked by priority — the primary set underpins the lab and next week; the secondary set goes deeper on the slides listed.

1 · Primary reading — read before the lab

1 Apache Kafka docs, *Design* (§4)
Persistence, delivery semantics, compaction · Slides 11–18

2 Narkhede et al., *Kafka: The Definitive Guide*, 2017
Ch. 3–4 producers & consumers, ch. 6 reliability · Slides 9–16

3 Confluent docs, *Schema Evolution and Compatibility*
Compatibility modes the registry enforces · Slides 19–20, 25–26

4 Kleppmann, *Designing Data-Intensive Applications*, 2017
Ch. 11 logs vs. brokers; due before week 8 · Slides 7–8

2 · Secondary reading — go deeper

1 Kreps et al., *Kafka* (original paper), NetDB 2011
Partitioned logs and pull-based consumers · Slides 6, 10

2 Kreps, *The Log*, LinkedIn Engineering, 2013
Why the log is the unifying abstraction · Slides 5–7

3 Kleppmann, *DDIA*, 2017, ch. 4
Encoding & evolution, backward/forward compatibility · Slides 19–20

4 Apache Kafka, *KIP-500: Replace ZooKeeper*, 2019
Why KRaft took over cluster metadata · Slide 22

5 Apache Kafka docs, *Kafka Connect & Kafka Streams*
Moving and processing streams, weeks 8–9 · Slide 22

6 Apache Kafka docs, *Quickstart*, Kafka 3.x
Bring up a broker, produce and consume · Lab 7, slide 25

Read the primary set in order; pick secondary items as your lab job needs.

PART 2 · HANDS-ON

Lab

07